

Task 3 — Sales Prediction Model

1. Project Title

Sales Prediction Model Using Machine Learning

2. Project Overview

This project demonstrates a complete machine learning regression pipeline using Python and Scikit-learn.

The objective of the task is to build machine learning models that can predict a continuous numerical value using historical data, compare multiple regression algorithms, evaluate their performance, and optimize the best-performing model.

Dataset Used

india_job_market_2024_2026(3).csv

Important Dataset Adaptation

The supplied dataset is an **India Job Market dataset** and does not contain a sales or revenue column.

Therefore, **Salary_LPA** is used as the prediction target for this regression project.

The same machine learning pipeline can be applied to a real sales dataset by replacing **Salary_LPA** with the appropriate sales/revenue column.

3. Objective

The main objectives of this project are:

- Load and explore the dataset.
- Perform data preprocessing.
- Engineer useful features.
- Handle missing values.
- Encode categorical variables.
- Split the dataset into training and testing sets.
- Build a Linear Regression baseline model.
- Build a Random Forest Regression model.
- Compare model performance.
- Calculate RMSE, MAE, and R^2 .
- Visualize feature importance.
- Visualize actual vs. predicted values.

- Perform hyperparameter tuning.
 - Select the best-performing model.
 - Interpret the results from a business perspective.
-

4. Dataset Description

The dataset contains approximately **5,000 job-market records**.

The dataset includes information related to:

- Job roles
- Companies
- Locations
- Industries
- Experience
- Education
- Employment type
- Salary
- Applicants
- Openings
- Job posting dates
- Other job-related attributes

Target Variable

Salary_LPA

This is the continuous numerical variable that the machine learning models predict.

5. Technologies Used

Programming Language

- Python 3.x

Libraries

- Pandas
- NumPy
- Scikit-learn
- Matplotlib

- Seaborn

Development Environment

- Jupyter Notebook
 - Google Colab
 - VS Code (optional)
-

6. Machine Learning Workflow

The project follows the complete machine learning pipeline:

Dataset



Data Loading



Data Exploration



Data Cleaning



Feature Engineering



Feature Selection



Categorical Encoding



Train-Test Split



Linear Regression



Random Forest Regression



Model Evaluation



Feature Importance



Hyperparameter Tuning



Best Model



Business Interpretation

7. Data Exploration

The following methods were used to understand the dataset:

`df.head()`

`df.info()`

`df.describe()`

`df.shape`

`df.isnull().sum()`

`df.duplicated().sum()`

These checks were used to identify:

- Number of rows and columns
 - Data types
 - Missing values
 - Duplicate records
 - Numerical distributions
 - Categorical variables
-

8. Data Preprocessing

Missing Values

Numerical variables are handled using median imputation.

Categorical variables are handled using the most frequent value.

`SimpleImputer(strategy="median")`

and:

`SimpleImputer(strategy="most_frequent")`

Categorical Encoding

Categorical variables are converted into numerical representations using:

```
OneHotEncoder(handle_unknown="ignore")
```

This allows machine learning algorithms to work with categorical features.

9. Feature Engineering

The Date_Posted column was converted into useful date-based features:

- Posting year
- Posting month
- Posting quarter
- Day of week
- Day of month

An additional ratio feature was created:

Applicants per Opening =

Applicants / Openings

Example

```
d["posted_year"] = d["Date_Posted"].dt.year
```

```
d["posted_month"] = d["Date_Posted"].dt.month
```

```
d["posted_quarter"] = d["Date_Posted"].dt.quarter
```

```
d["posted_dayofweek"] = d["Date_Posted"].dt.dayofweek
```

```
d["posted_dayofmonth"] = d["Date_Posted"].dt.day
```

10. Feature Selection

The following fields were excluded from the baseline model:

Job_ID

This is an identifier and does not provide useful predictive information.

Date_Posted

The original date was replaced by engineered date features.

Skills_Required

This is a free-text field. It was excluded from the baseline model because processing it properly would require NLP techniques such as TF-IDF or text embeddings.

11. Train-Test Split

The dataset was divided into:

- **80% Training Data**
- **20% Testing Data**

The split was performed using:

```
train_test_split(  
    X,  
    y,  
    test_size=0.20,  
    random_state=42  
)
```

The training data is used to build the models, while the testing data is used to evaluate their performance on unseen records.

12. Machine Learning Models

Model 1 — Linear Regression

Linear Regression was selected as the baseline model.

It provides a simple benchmark for evaluating more advanced models.

```
LinearRegression()
```

Model 2 — Random Forest Regression

Random Forest was selected because it can capture:

- Non-linear relationships
- Feature interactions
- Complex patterns
- Relationships between categorical and numerical variables

The model was implemented using:

```
RandomForestRegressor(  
    n_estimators=200,  
    random_state=42  
)
```

13. Model Evaluation Metrics

Three evaluation metrics were used.

RMSE — Root Mean Squared Error

RMSE measures the average magnitude of prediction errors while giving more weight to larger errors.

Lower RMSE is better.

MAE — Mean Absolute Error

MAE represents the average absolute difference between actual and predicted values.

Lower MAE is better.

R^2 — R-squared

R^2 measures how much variation in the target variable is explained by the model.

Higher R^2 is better.

14. Model Performance

The models achieved the following results on the test dataset:

Model	RMSE	MAE	R^2
Linear Regression	8.0688	5.3302	0.7964
Random Forest	4.3523	2.7090	0.9408
Tuned Random Forest	4.3425	2.7137	0.9410

15. Best Performing Model

The **Tuned Random Forest Regression model** achieved the strongest overall performance.

Performance

- **RMSE:** 4.3425
- **MAE:** 2.7137
- **R^2 :** 0.9410

The Random Forest models performed considerably better than Linear Regression, indicating that the relationships in the dataset are not purely linear.

16. Hyperparameter Tuning

The Random Forest model was optimized using:

RandomizedSearchCV

The parameters explored included:

- Number of trees
- Maximum tree depth
- Minimum samples per leaf
- Maximum features

Example:

```
RandomizedSearchCV(  
    estimator=model,  
    param_distributions=params,  
    cv=2,  
    scoring="neg_root_mean_squared_error",  
    random_state=42  
)
```

The purpose of hyperparameter tuning was to identify a configuration that improved predictive performance while reducing overfitting.

17. Visualizations

The project includes the following visualizations.

1. Model Comparison

Compares:

- RMSE
- MAE
- R^2

between the regression models.

2. Feature Importance

Shows the most influential features used by the Random Forest model.

3. Actual vs Predicted

A scatter plot compares actual salary values with model predictions.

A model with strong predictive performance should have points close to the diagonal reference line.

18. Business Interpretation

The machine learning model can be used as an exploratory salary prediction tool.

For example, organizations or job-market analysts could use job characteristics such as:

- Experience
- Industry
- Job role
- Location
- Education
- Company characteristics
- Number of applicants
- Number of openings

to estimate expected salary levels.

The Random Forest model performed substantially better than the Linear Regression baseline, suggesting that salary relationships in this dataset are complex and nonlinear.

19. Key Findings

Finding 1

Random Forest significantly outperformed Linear Regression.

Finding 2

The tuned Random Forest achieved an R^2 of approximately **0.941**, indicating strong predictive performance on the test dataset.

Finding 3

The model's lower RMSE demonstrates that its predictions were considerably closer to actual salary values than those produced by the linear baseline.

Finding 4

Feature importance analysis helps identify which job-related characteristics contribute most to the model's salary predictions.

20. Limitations

This project has several limitations.

1. Dataset Type

The supplied dataset is an India Job Market dataset rather than a traditional sales dataset.

Therefore:

Salary_LPA

was used as the regression target.

2. Free-Text Skills

The Skills_Required column was not included in the baseline model because it contains text.

An advanced project could use:

- TF-IDF
- Word embeddings
- NLP
- Sentence Transformers

to extract information from this column.

3. Dataset Quality

The model's performance depends on the quality and representativeness of the supplied dataset.

4. Prediction Does Not Mean Causation

Feature importance indicates relationships used by the model. It does not prove that a feature directly causes higher or lower salaries.

5. Sales Forecasting

For an actual sales forecasting project, a chronological train-test split should generally be used instead of a random split.

21. How to Run the Project

Step 1 — Install Python

Install Python 3.x.

Step 2 — Install Required Libraries

Run:

```
pip install pandas numpy scikit-learn matplotlib seaborn jupyter
```

Step 3 — Open Jupyter Notebook

Run:

```
jupyter notebook
```

Step 4 — Open the Notebook

Open:

Task_3_Sales_Prediction_Model.ipynb

Step 5 — Place the Dataset

Make sure the dataset is in the same directory as the notebook:

india_job_market_2024_2026(3).csv

Step 6 — Run All Cells

Run the notebook from the first cell to the last cell.

22. Project Folder Structure

Task_3_Sales_Prediction/

|

├─ india_job_market_2024_2026(3).csv

|

├─ Task_3_Sales_Prediction_Model.ipynb

|

├─ README.md

|

├─ Task_3_Model_Performance_Summary.md

|

└─ task3_sales_prediction_outputs/

|

├─ model_comparison.png

├─ actual_vs_predicted.png

├─ feature_importance.png

|

├─ model_comparison_baseline.csv

├─ model_comparison_final.csv

└─ top_feature_importance.csv

23. Future Improvements

The project can be improved by:

- Using a real sales/revenue dataset
 - Adding lag features
 - Adding rolling averages
 - Using chronological validation
 - Applying XGBoost or Gradient Boosting
 - Performing more extensive hyperparameter tuning
 - Applying NLP to Skills_Required
 - Using cross-validation
 - Deploying the model using Streamlit or Flask
 - Creating an interactive prediction dashboard
-

24. Conclusion

This project successfully demonstrates a complete machine learning regression workflow.

The project includes:

- Data exploration
- Data preprocessing
- Feature engineering
- Feature selection
- Categorical encoding
- 80/20 train-test split
- Linear Regression
- Random Forest Regression
- RMSE evaluation
- MAE evaluation
- R^2 evaluation
- Feature importance visualization
- Actual vs predicted visualization
- Hyperparameter tuning
- Business interpretation

The **Tuned Random Forest** was the best-performing model with an **R^2 of 0.9410** and an **RMSE of 4.3425**.

For a true sales prediction assignment, the same pipeline can be reused with a sales/revenue target and appropriate time-series features.

25. Author Details

Name: Megha Bidwe

Course/Internship: Internship

Project: Task 3 — Sales Prediction Model

Technology: Python / Machine Learning